# larpix-lib on PACMAN

LArPix control software: architecture, setup, bench results and status

Maintained by the PACMAN agents (augie)

2026-11-05

## 1 Purpose and scope

`larpix-lib` (package `larpix_control`) is the new LArPix control software for PACMAN with the zipline firmware and software (`zipline.tex`). Since 2026-11-05 it is a tool Michael supports officially, no longer an experiment, and this record moved back into pacman_docs `agents/` beside `zipline.tex`. The goal is for `larpix-lib` to replace (1) the control path, including readback for control, and (2) the data server; the focus is (1). The ASICs themselves (bench setup, registers, TX settings) are recorded in `larpix.tex`.

Scope (Michael 2026-11-05): a single Hydra strand on one UART channel, taken far; more than one channel is out of scope for now (the binding of io groups and channels to strands is postponed), but the design must scale to the whole system. Every PACMAN carries precisely one ASIC type, so the ASIC version is an attribute of the PACMAN.

## 2 Repository and environment

- `mulhearn/larpix-lib`, cloned in `/agents/cowork/larpix-lib` (shared repository; Michael pushes). Work is on the branch `working` (no releases yet; `main` follows `origin/main`). At c78825f: PEP 8 names (e1e2dbb), David Yang's authorship commit (fea7566), the closed loop (322c119), board control and `demos/hello_asic.py` (c78825f).
- Python environment: conda environment `larpix` per user, an editable install of the clone; `make test` runs the unit tests (41, OK).
- Names (Michael 2026-11-05): Python classes in CapWords (PEP 8), functions, methods, modules snake_case, constants UPPER_CASE. "PACMAN" names only the real board or an accurate emulation; toys and generic contracts say "board".

## 3 Architecture

`larpix_control` separates what varies independently: the ASIC version, what each configuration step means, the network topology, register access to the ASICs, and control of the board they sit on. Version-specific content lives in YAML, not code. The contracts are abstract base classes in `common.interfaces`.

| Layer | Module | Role |
|---|---|---|
| ASIC spec | `asic.asic_spec` | One ASIC version from YAML (`config/asics/larpix_v3.yaml`): register space and config packet layout. Field updates to register writes (`build_write_list`), registers to config packets, and the read side (`valid_config_read_response`, `parse_chip_address_value`). No I/O. |
| Fragments | `asic.fragment_lib` | Named configuration steps in YAML (`config/fragments/`) with external parameters and per-version overrides. |
| ASIC config | `AsicConfigIface`, `asic.asic_config` | The operations a network needs (chip ID, I/O init, input and output enables), one step up from register access. `AsicConfig` evaluates a fragment, turns the field updates into `RegisterOp` writes on its io channel and transmits them. |
| Topology | `hydra.strand` | One Hydra network on one UART, from YAML (`config/hydra/`); configures it from the root outward. Uses only `AsicConfigIface`. |
| Register access | `AsicRegisterIface` | `transmit(ops, fire_and_forget)`: `RegisterOps` (`OpType` WRITE or READ, `nrep` repeats, io channel, chip, register, value) in order; returns `OpReplies`: per register (`RegisterKey`) the `WriteReply` (requested, acknowledged) and `ReadReply` (requested, acknowledged, majority value, received, discrepant), and an overall `complete`. |
| Board control | `BoardControlIface` | One readout board (one io group): `ping`, `get_tile`, `reset` (FULL or INTERNAL, by tile) with `wait_on_reset`, `power_on`/`power_off` (by tile), `uart_enable`/`uart_disable` (by io channel). Pokes return at once, without replies. |
| PACMAN | `pacman.pacman` | `Pacman` implements both: one PACMAN from one network file entry; ASIC words and board register writes through `pacman_cmdserver` (each acknowledged), replies from `pacman_dataserver` matched by the reply correlator (`asic._correlator`). `load_network` builds one per entry; `sync_pacman_id` sets the PACMAN's own ID to its io group. |
| Toy | `common.board_toy` | `BoardToy` implements both without hardware: registers in memory, resets, power and UART enables with their effect on the chips. |

A configuration runs: `Strand.init_network` → `AsicConfig` operation → fragment → field updates → register writes → `RegisterOps` → `Pacman.transmit` → config packets → `pacman_cmdserver`; replies return through `pacman_dataserver` to the correlator and the `OpReplies`.

### 3.1 Design decisions (Michael 2026-11-05)

- Chip identity is `(io_chan, chip_id)`: chip IDs are unique only within an io channel. Each board owns a configured range of io channels (`first_io_chan`, `uart_mask`); the upper bits may encode the PACMAN by configuration, never checked. The wire channel is the io channel's low byte (UART + 1); the UART is `io_chan - first_io_chan`, so no "+1" appears in our code.
- One board per io group: the board knows its io group, so board control takes tiles and io channels only.
- Replies: one persistent SUB socket per PACMAN, subscribed only during a transmission (an idle subscriber cannot stall the data server) and drained at its start (no stale reply counts as fresh). The race between the subscription and the first reply is accepted until the 3.7 XPUB confirmation.
- The timeout is a property of each implementation (`timeout_ms` in the network file, 200 ms), bounding the acknowledgements and the wait for replies. Missing acknowledgements and replies are reported, not raised. `fire_and_forget` skips collecting replies; acknowledgements are still counted.

- Reads repeated (`nrep` or listed again) are collated: the majority value (the last received on a tie) and the number discrepant.
- PACMAN registers, power values and the reset pokes are private to `Pacman`, from `pacman_vspace.cc`. The code there sends the internal reset at 0x00100410 and the full reset at 0x00100420; its header comment has the two swapped.
- Interface first, in Python: the interface step reviews generated `.pyi` stubs (baseline committed on the interface branch, the draft in the stubs, propagated into the `.py` once accepted); the stubs never reach `working`.

## 4 Network file

`config/network/*.yaml`, one entry per PACMAN:

| Field | Meaning |
|---|---|
| `io_group` | The PACMAN; set as its own ID by `sync_pacman_id`. |
| `hostname`, `port` | `pacman_cmdserver` (5555). |
| `data_port` | `pacman_dataserver`; optional, default 5556. |
| `timeout_ms` | Acknowledgement and reply timeout (200). |
| `first_io_chan`, `uart_mask` | The board's io channels: `first_io_chan` + $u$ for each bit $u$; the low byte of each must be its PACMAN channel. |
| `uarts_per_tile` | 4 on every revision (1v5: 10 tiles, 40 UARTs; 1v4: 8 tiles, 32 UARTs). |

The bench file is `single_bench.yaml`: the 1v5 at 10.42.0.175, io group 1, `first_io_chan` 0x001, `uart_mask` 0xF (tile 0, io channels 1–4).

## 5 Hello ASIC

`demos/hello_asic.py`, from the repo top, takes a PACMAN just booted or powered up to a configured, answering strand:

1. Ping until `pacman_cmdserver` answers (up to `-wait`, 120 s), then `sync_pacman_id`.
2. On the strand's tile: UART off, `power_on`, 50 ms, full reset, `wait_on_reset` (1 ms).
3. Configure the strand (`Strand` → `AsicConfig` → `Pacman`), recording every register written; UART on.
4. Read back every register written and each chip's ID register; PASS or FAIL, and the exit status.

Options: `-network`, `-io_group`, `-io_chan`, `-hydra`, `-asic`, `-wait`, `-v` (the configuration trace and the board's warnings). No RX buffer enables are written: the correlator ignores everything but config replies. It replaces the earlier `agents/bringup.sh` and `pacman_util.py`.

The settles come from a sweep on the 1v5 (tile 0 off 1 s before each run, 10 runs per step): a full reset passes with no settle at all (the reset is about 13 µs, shorter than the next round trip), and the power settle passes from 10 ms and fails at 5 ms and below. 50 ms and 1 ms keep margins of 5× and about 75×; the 1v4 and other tiles are not measured.

## 6 Results

| Date | Board | Result |
|---|---|---|
| 2026-11-02 | 1v5 | First configuration by `larpix-lib` (`demos/init_network.py` from felix, through `pacman_cmdserver`), with the input enables left out: 13 writes. Then a menu hello without the configuration root: the reply exact, 3 of 3. Before it, after a full reset, hello had no reply. |
| 2026-11-02 | 1v5 | With the input enables written (126 = 0x02, POSI1 only; 14 writes): the reply exact, 3 of 3; read back 124 = 0x00, 125 = 0x01, 126 = 0x02, 248 = 0x07. UART 0 drives POSI1 (west), as the port map has it. |
| 2026-11-02 | 1v5 | The full configuration sequence (fce6523, "ASIC config working at test bench"): the reply exact, 3 of 3; io channels 2–4 unanswered, channel 1 exact. |
| 2026-11-03 | 1v5 | The whole chain, by Michael, on the software release 94e3f01: the configuration by `larpix-lib` through `pacman_cmdserver`, the menu's hello replies seen on felix through the data server. |
| 2026-11-04 | 1v5 | Bring-up from felix alone (then `agents/bringup.sh`): tile 1 power (VDDA 2367 mV, VDDD 1302 mV), full reset, 14 configuration writes, read-back through the data server. One read at a time exact; four in a row, only the first and last replies: the data server's PUB socket dropped them (send high-water mark 1). |
| 2026-11-04 | 1v5 | `PUB_HWM` 1000: four reads in a row 4 of 4, 64 in a row 64 of 64; `pacman_push` 20000 words, all counted (6680 with HWM 1). In software 3.6.2. |
| 2026-11-05 | 1v5 | The closed loop (`AsicRegisterIface`, `PacmanAsicRegister` then): 4 reads answered in 6 ms (was a fixed 0.5 s subscription sleep); `nrep` 3 without discrepancy; 64 registers in one transmission in 8 ms; fire and forget acknowledged with no reply collected and nothing stale after it; pixel trim 0 written and read back, then restored; `init_network.py` through `AsicConfig`, configuration unchanged; 20 of 20 runs. |
| 2026-11-05 | 1v5 | Board control (`Pacman`): `read_pacman_id` 1 after `sync_pacman_id`; with UART 1 disabled a read is acknowledged with no reply (`complete` False), answered again once enabled; an internal reset of tile 0 keeps every register; a full reset silences chip 11 and `init_network.py` restores it from chip 1; tile 0 power off and on: VDDA 2365 → 29 → 2367 mV, VDDD 1302 → 8 → 1302 mV. |
| 2026-11-05 | 1v5 | `hello_asic.py`: PASS warm (14 of 14 registers, chip ID included); after `reboot`, PASS 29 s after the reboot (the board answered after about 22 s); after a supply power cycle (halt, `OUTP 0`, `OUTP 1`), PASS 47 s after power-on. With `timeout_ms` 200 and the swept settles: 0.18–0.19 s, warm and from a powered-off tile (6 of 6). |

## 7 Student work

David Yang (`dy241/larpix-lib`, 2026-06 to 08): his reply correlator and `AsicSpec` read-side fixes are in `larpix-lib` as his own commit (fea7566, authored with his GitHub no-reply address; an authorship commit, not in a working state on its own), adapted to `AsicRegisterIface` in the next commit. His data packet format and his ASIC simulator with multi-chip grids (`sim/`) are not taken yet; the simulator would implement both interfaces.

## 8 Open

- Chip state: the expected registers per chip, readback verification, the other fields of partly written registers (`build_write_list`'s `as_needed`), the `CONFIRMED_WRITE` and `WRITE_READ` op types.
- Pedestals (tabled): data packets in the ASIC YAML, a trigger fragment, per-channel mean and RMS.
- Board control: the power monitors and other replies (deferred: pokes return nothing for now); whether `power_on` alone leaves the ASICs as after a full reset (untested: `hello_asic.py` always

resets).

- Deployment variations and more than one channel or board: the binding of io groups and channels to strands.
- For 3.7: the XPUB subscription confirmation, separate config and data streams, polling for reset complete.